COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/2026	**DATE SUBMITTED:**	09/23/2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **TREE ADT**

1. PROCEDURE OUTPUT

GenTree.cpp

```cpp
#include <iostream>
#include "GeneralTree.h"

//print tree structure
void printTree(GeneralTree<char>* node, int depth = 0) {
    if (node == nullptr) return;
    // Print current node with indentation
    for (int i = 0; i < depth; ++i) std::cout << "  ";
    std::cout << node->getRootVal() << std::endl;

    // Print first child
    printTree(node->getLeftChild(), depth + 1);

    // Print next sibling
    printTree(node->getRightChild(), depth);
}

int main() {
    // Create Root Node 'A'
    GeneralTree<char>* A = new GeneralTree<char>('A');

    //Attach Children to A
    A->insertLeftChild('B');
    GeneralTree<char>* B = A->getLeftChild();

    B->insertRightChild('C');
    GeneralTree<char>* C = B->getRightChild();

    C->insertRightChild('D');
    GeneralTree<char>* D = C->getRightChild();

    D->insertRightChild('E');
    GeneralTree<char>* E = D->getRightChild();

    E->insertRightChild('F');
    GeneralTree<char>* F = E->getRightChild();

    F->insertRightChild('G');
    GeneralTree<char>* G = F->getRightChild();

    //Insert 2nd and 3rd row
    D->insertLeftChild('H');

    E->insertLeftChild('I');
    GeneralTree<char>* I = E->getLeftChild();
    I->insertRightChild('J');
    GeneralTree<char>* J = I->getRightChild();

    J->insertLeftChild('P');
    GeneralTree<char>* P = J->getLeftChild();
    P->insertRightChild('Q');

    F->insertLeftChild('K');
    GeneralTree<char>* K = F->getLeftChild();
    K->insertRightChild('L');
    GeneralTree<char>* L = K->getRightChild();
    L->insertRightChild('M');

    G->insertLeftChild('N');

    //print general tree
    std::cout << "General Tree Structure: " << std::endl;
    printTree(A);

    //destructor
    delete A;

    return 0;
}
```

Figure 1 & 2. Code Snippet for GenTree.cpp

GeneralTree.H

```h
#ifndef GENERALTREE_H
#define GENERALTREE_H

template <typename T>
class GeneralTree {
private:
    T key;
    // PointersA
    GeneralTree<T>* firstChild;
    GeneralTree<T>* nextSibling;

public:
    // Constructor:
    GeneralTree(T rootObj);

    // Destructor
    ~GeneralTree();

    // Inserting a child node
    void insertLeftChild(T newNode);
    void insertRightChild(T newNode);

    GeneralTree<T>* getLeftChild();
    GeneralTree<T>* getRightChild();

    T getRootVal();

    void setRootVal(T obj);

    bool isleaf();
};

template <typename T>
GeneralTree<T>::GeneralTree(T rootObj) {
    key = rootObj;
    firstChild = nullptr;
    nextSibling = nullptr;
}

template <typename T>
GeneralTree<T>::~GeneralTree() {
    delete firstChild;
    delete nextSibling;
}

template <typename T>
void GeneralTree<T>::insertLeftChild(T newNode) {
    if (firstChild == nullptr) {
        firstChild = new GeneralTree<T>(newNode);
    } else {
        //push existing firstChild to nextSibling
        GeneralTree<T>* newChild = new GeneralTree<T>(newNode);
        newChild->nextSibling = firstChild;
        firstChild = newChild;
    }
}
```

```h
template <typename T>
void GeneralTree<T>::insertRightChild(T newNode) {
    if (nextSibling == nullptr) {
        nextSibling = new GeneralTree<T>(newNode);
    } else {
        //push existing nextSibling down
        GeneralTree<T>* newChild = new GeneralTree<T>(newNode);
        newChild->nextSibling = nextSibling;
        nextSibling = newChild;
    }
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getLeftChild() {
    return firstChild;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getRightChild() {
    return nextSibling;
}

template <typename T>
T GeneralTree<T>::getRootVal() {
    return key;
}

template <typename T>
void GeneralTree<T>::setRootVal(T obj) {
    key = obj;
}

template <typename T>
bool GeneralTree<T>::isleaf() {
    return firstChild == nullptr && nextSibling == nullptr;
}

#endif
```

Figure 3 & 4. Code Snippet for GeneralTree.H

GenTree.cpp shows the exact application of trees using the templating that we did in the binary trees. This implementation of the main file allowed me to recreate the general tree structure that the image in the lab shown. As

you can see, I created a root node A and attached children into it. First, at the left, which is the B child, then I just continued to insert values in the right node until all children are listed. I then repeated the same process for nodes with multiple children.

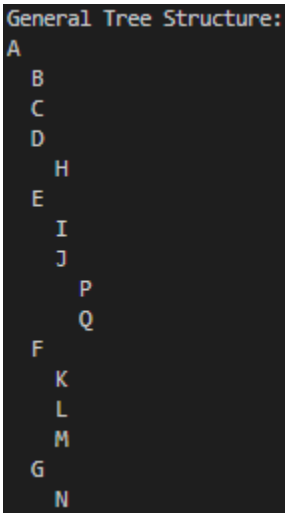


Figure 5. Output

As you can see in the output, the Parent node at the top most part, shows that it is the topmost part of the tree, which is then followed by its children and so on. In this output, the representation is only in vertical form which is why it is confusing.

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2

M	0	2
N	0	2
P	0	3
Q	0	3

Table 1

ILOB

Pre-Order Traversal	A B C D H E I J P Q F K L M G N
In-Order Traversal	B C H D I P Q J E K L M F N G A
Post-Order Traversal	H Q P J I M L K N G F E D C B A

Table 2

```
// TREE TRAVERSAL FUNCTIONS

//PRE-ORDER TRAVERSAL
template <typename T>
void preOrder(GeneralTree<T>* node) {
    if (node == nullptr) return;

    std::cout << node->getRootVal() << " ";

    preOrder(node->getLeftChild());

    preOrder(node->getRightChild());
}

//IN-ORDER TRAVERSAL
template <typename T>
void inOrder(GeneralTree<T>* node) {
    if (node == nullptr) return;

    inOrder(node->getLeftChild());

    std::cout << node->getRootVal() << " ";

    inOrder(node->getRightChild());
}

//POST-ORDER TRAVERSAL
template <typename T>
void postOrder(GeneralTree<T>* node) {
    if (node == nullptr) return;

    postOrder(node->getLeftChild());

    std::cout << node->getRootVal() << " ";

    postOrder(node->getRightChild());
}
```

Figure 6. Code Snippet for Traversal Functions

Pre-Order takes the root value, then moves to the first child, then also goes down to traverse on its child until it reaches the deepest node before going back up and moving to the next sibling. In order goes straight to the first

child, until it goes down to the deepest leftmost node, prints that node and goes back to the parent node and proceeds to print everything horizontally. And post-order, goes all the way to the deepest node, finishes all the left side of the nodes first, then the right side before the parent gets printed out.

```
Pre-Order Traversal:  A B C D H E I J P Q F K L M G N

In-Order Traversal:   B C H D I P Q J E K L M F N G A

Post-Order Traversal: H Q P J I M L K N G F E D C B A
```

Figure 7. Output for traversal functions

```cpp
template <typename T>
bool findDataHelper(GeneralTree<T>* node, const std::string& choice, T key) {
    if (node == nullptr) return false;

    // PRE-ORDER
    if (choice == "PRE" || choice == "pre") {
        if (node->getRootVal() == key) return true;
        if (findDataHelper(node->getLeftChild(), choice, key)) return true;
        if (findDataHelper(node->getRightChild(), choice, key)) return true;
    }
    // IN-ORDER
    else if (choice == "IN" || choice == "in") {
        if (findDataHelper(node->getLeftChild(), choice, key)) return true;
        if (node->getRootVal() == key) return true;
        if (findDataHelper(node->getRightChild(), choice, key)) return true;
    }
    // POST-ORDER
    else if (choice == "POST" || choice == "post") {
        if (findDataHelper(node->getLeftChild(), choice, key)) return true;
        if (findDataHelper(node->getRightChild(), choice, key)) return true;
        if (node->getRootVal() == key) return true;
    }

    return false;
}
```

Figure 8. Helper functions and Search functions

Validation for the pre-order, in-order and post-order functions. It acts as the main searching function on how the traversion finds the data or the exact node in the trees using the three traversal techniques.

```
Pre-Order Traversal:  A B C D H E I J P Q F K L M G N O

In-Order Traversal:   B C H D I P Q J E K L M F N O G A

Post-Order Traversal: H Q P J I M L K O N G F E D C B A
```

Figure 9. Output for traversal functions

```
std::cout << "Find Results" << std::endl;
findData<char>(A, "PRE", 'E');
findData<char>(A, "IN", 'E');
findData<char>(A, "POST", 'E');

findData<char>(A, "PRE", 'O');
findData<char>(A, "IN", 'O');
findData<char>(A, "POST", 'O');
```

Figure 10. Code snippet for Find data

```
template <typename T>
void findData(GeneralTree<T>* root, std::string choice, T key) {
    if (findDataHelper(root, choice, key)) {
        std::cout << key << " was found!" << std::endl;
    }
}
```

Figure 11. Find data validation

Acts as the validator if a key was found in the tree structure.

```
Find Results
E was found!
E was found!
E was found!
O was found!
O was found!
O was found!
```

Figure 12. Output for find data

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key;
        //pointers
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        //constructor:
        BinaryTree(T rootObj);

        //inserting a child node
        void insertLeftChild(T newNode);
        void insertRightChild(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();

        void setRootVal(T obj);

        bool isleaf();
};

template <typename T>
BinaryTree<T>::BinaryTree(T rootObj){

    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;

}

template <typename T>
void BinaryTree<T>::insertLeftChild(T newNode){
    leftChild = new BinaryTree<T>(newNode);

    if (leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    }else{
    //Create a new Node
    BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

    newChild ->leftChild = leftChild;
    leftChild = newChild;
    }
}
```

```
template <typename T>
void BinaryTree<T>::insertRightChild(T newNode){
    rightChild = new BinaryTree<T>(newNode);

    if (rightChild == nullptr){
        rightChild = new BinaryTree<T>(newNode);
    }else{
    //Create a new Node
    BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

    newChild ->rightChild = rightChild;
    rightChild = newChild;
    }
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild(){
    return rightChild;
}

template<typename T>
T BinaryTree<T>::getRootVal(){
    return key;
}

template<typename T>
void BinaryTree<T>::setRootVal(T obj){
    key = obj;
}

template<typename T>
bool BinaryTree<T>::isleaf(){
    return leftChild == nullptr && rightChild == nullptr;
}
#endif
```

Figure 13 and 14. Code snippet for BinaryTree.H

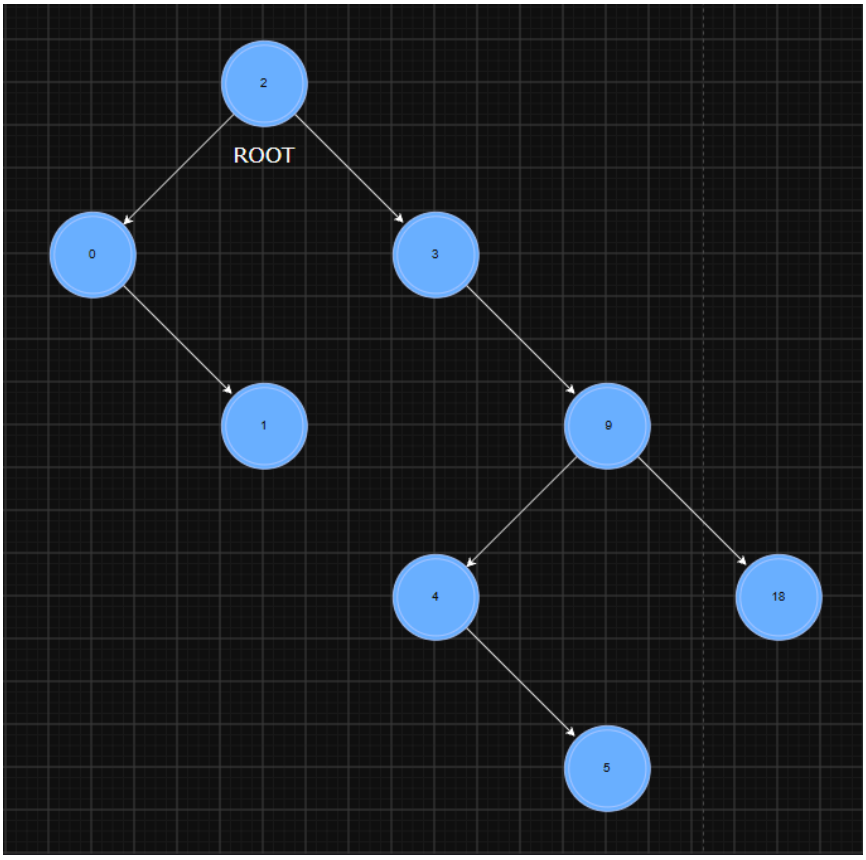


Figure 15. Tree Structure

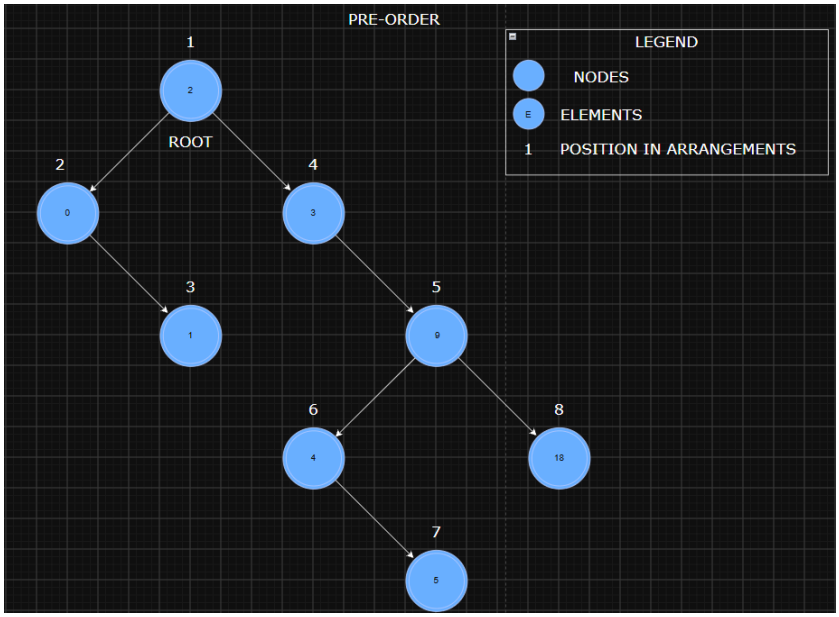


Figure 16. Pre-Order Traversal

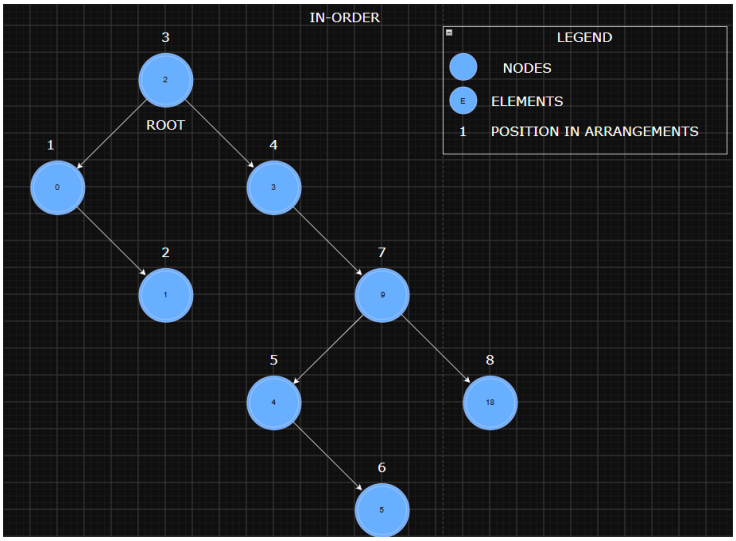


Figure 17. In-Order Traversal

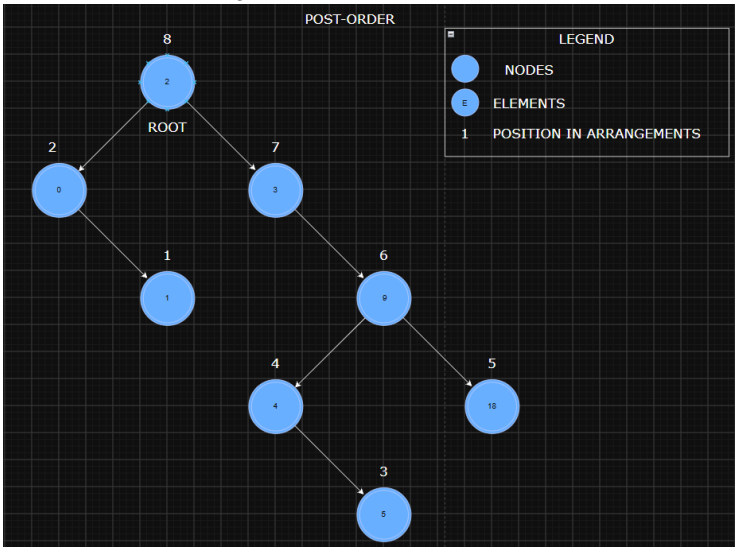
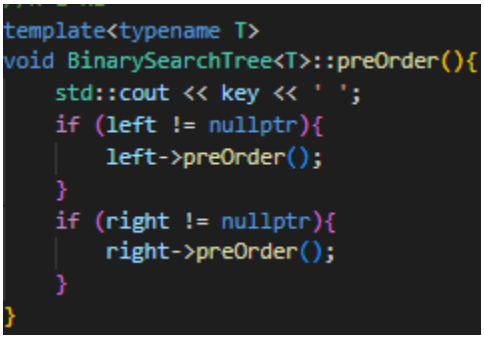


Figure 18. Post-Order Traversal

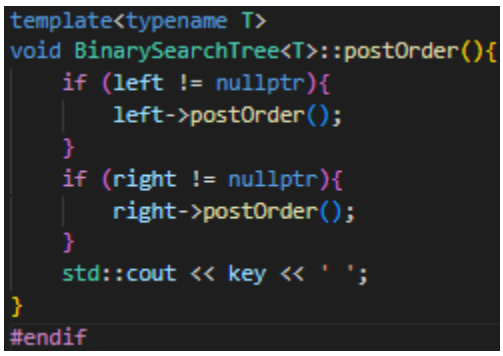
```
template<typename T>
void BinarySearchTree<T>::inOrder(){
    if (left != nullptr){
        left->inOrder();
    }
    std::cout << key << ' ';
    if (right != nullptr){
        right->inOrder();
    }
}
```

Figure 19. In-Order Traversal



```
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << ' ';
    if (left != nullptr){
        left->preOrder();
    }
    if (right != nullptr){
        right->preOrder();
    }
}
```

Figure 20. Pre-Order Traversal



```
template<typename T>
void BinarySearchTree<T>::postOrder(){
    if (left != nullptr){
        left->postOrder();
    }
    if (right != nullptr){
        right->postOrder();
    }
    std::cout << key << ' ';
}
#endif
```

Figure 21. Post-Order Traversal

Analysis:

From figure 19 - 21, the tree traversal are executed if the leaf is not equal to null, which is why it also acts as a helper function other than executing the code itself or the traversion itself.

Conclusion:

Overall, I learned about how trees work and how traversion occurs in a tree structure. Basically, a tree is a large data structure that you can use to traverse in a hierarchy manner depending on what you need. You can perform pre-order, post-order and in-order as long as you know the implementation and the concept behind it. Overall, I think that I've learned a lot in this topic, though I still haven't mastered it completely and I am not yet confident in my skill, I think that I can do better if I practice hard and understand the fundamentals of managing and manipulating data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.